[CS639] HW4 Fine-Tuning Assignment

Due date: April 22, 2026 11:59PM

P.S. It may be tempting to use an LLM to generate solutions for this assignment, and it would probably save you time. But if you're considering ML-focused roles after graduation, the concepts and implementation skills in this homework are what technical interviews usually test. Please also be aware that submissions may be checked for LLM-generated content. Cite any LLM based code or writeup and provide the link to LLM chat at the start of writeup PDF.

1 Baseline Full Fine-Tuning

1.1 Setup

The NLI dataset we will use for this assignment is SNLI, loaded via the HuggingFace `datasets` library. Each example has a `premise`, a `hypothesis`, and a `label` ($0 =$ entailment, $1 =$ neutral, $2 =$ contradiction). Examples with label -1 (annotator disagreement) should be filtered out.

The primary model is DistilBERT (`distilbert-base-uncased`), but you may use any small transformer: BERT-base, RoBERTa-base, ALBERT-base, or DeBERTa-small. Install the required libraries before you begin:

A single GPU is sufficient for all tasks. Google Colab (free T4 GPU) works well.

1.2 [Code] Perform Task (5 points)

Load and filter the SNLI dataset. Tokenize premise–hypothesis pairs using the tokenizer for your chosen model with `truncation=True` and `max_length=128`.

Load the model with `AutoModelForSequenceClassification` (`num_labels=3`). Train for 3 epochs, batch size 32, warmup ratio 0.06, and weight decay 0.01. Evaluate at the end of each epoch.

1.3 [Code, Writeup] Analysis and Plots (5 points)

Run predictions on the test set and report the test accuracy. Generate and plot a confusion matrix across all three labels using `seaborn` or `matplotlib`. Plot the training loss and dev accuracy curves across epochs. Report per-class precision, recall, and F1 scores for entailment, neutral, and contradiction.

1.4 [Writeup] Comparative Study (5 points)

Compare the per-class metrics. Which class does the model handle best? Which is hardest? How do the confusion patterns relate to the linguistic properties of the three label categories?

1.5 [Writeup] Possible Reasons Behind Observations (5 points)

1. Which pair of labels is most frequently confused? Why might that be the case linguistically?
2. Do you see signs of overfitting in your loss/accuracy curves? What evidence supports your conclusion?
3. Why might the neutral class be harder to classify than entailment or contradiction?

2 Hyperparameter Sensitivity Study

2.1 Setup

In this task you will study how key hyperparameters affect fine-tuning performance. You will vary one hyperparameter at a time while holding all others at the baseline values from Task 1.

2.2 [Code] Perform Task (5 points)

Choose **three** hyperparameters from the table below. For each, test all of the suggested values for that selected hyperparameter while keeping all other settings at the Task 1 baseline.

Hyperparameter	Suggested Values
Learning rate	1×10^{-5}, 2×10^{-5}, 5×10^{-5}, 1×10^{-4}
Batch size	16, 32, 64
Epochs	1, 2, 3, 5
Warmup ratio	0.0, 0.06, 0.10
Weight decay	0.0, 0.01, 0.1

Important: You must re-initialize the model from the pre-trained checkpoint before every run. If you forget this, the second run starts from the first run's trained weights, making your comparison invalid.

This requires 9-12 training runs. Plan your compute time accordingly.

```
# Example for one of the hyperparameter:
learning_rates = [1e-5, 2e-5, 5e-5, 1e-4]
results = []

for lr in learning_rates:
    # CRITICAL: reload model from scratch each time
    model = AutoModelForSequenceClassification.from_pretrained(
        "distilbert-base-uncased", num_labels=3)
    args = TrainingArguments(
        output_dir=f"./results_lr_{lr}",
        learning_rate=lr,
        # ... keep other args same as Task 1
    )
    trainer = Trainer(model=model, args=args, ...)
    trainer.train()
    metrics = trainer.evaluate()
    results.append({"lr": lr, "acc": metrics["eval_accuracy"]})
```

2.3 [Code, Writeup] Analysis and Plots (5 points)

Create a results table listing every configuration and its dev accuracy. Create one plot per hyperparameter: hyperparameter value on the x-axis, dev accuracy on the y-axis. Use a log scale for learning rate. Note any runs where training diverged.

2.4 [Writeup] Comparative Study (5 points)

Rank the three hyperparameters by their impact on performance. Compare the best and worst configurations: how large is the accuracy gap? Based on your results, what hyperparameter setting would you recommend and why?

2.5 [Writeup] Possible Reasons Behind Observations (5 points)

1. Why does a learning rate that is too high cause training to fail? Explain in terms of the loss landscape.
2. Why might a larger batch size sometimes lower accuracy, even though it provides a more stable gradient estimate?
3. If increasing epochs beyond 3 hurts dev accuracy, what phenomenon is occurring and how would you mitigate it?

3 Parameter-Efficient Fine-Tuning (PEFT)

3.1 Setup

In this task you will compare full fine-tuning against parameter-efficient methods. You will use the `peft` library to apply LoRA and at least one other method.

3.2 [Code] Perform Task (5 points)

Step 1. Record the total trainable parameters of your fully fine-tuned model from Task 1.

```
total = sum(p.numel() for p in model.parameters()
            if p.requires_grad)
print(f"Full fine-tuning: {total:,} trainable parameters")
```

Step 2. Apply LoRA. Load a fresh pre-trained model and wrap it with a LoRA configuration. Train and evaluate. Repeat with ranks $r \in \{4, 8, 16\}$.

Step 3. Apply a second PEFT method: BitFit (bias-only tuning) or Prefix Tuning. For BitFit:

```
model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased", num_labels=3)
for name, param in model.named_parameters():
    if "bias" not in name:
        param.requires_grad = False
```

Train and evaluate with the same setup.

3.3 [Code, Writeup] Analysis and Plots (5 points)

Fill in the comparison table below and plot accuracy vs. trainable parameters (log scale on x-axis) for all methods on a single chart.

Method	Trainable Params	% of Total	Dev Acc	Train Time
Full fine-tuning				
LoRA (r=4)				
LoRA (r=8)				
LoRA (r=16)				
BitFit / Prefix				

3.4 [Writeup] Comparative Study (5 points)

At what LoRA rank does accuracy approach full fine-tuning? How does BitFit compare to LoRA in the accuracy–parameter trade-off? In what practical scenarios (memory constraints, multi-task serving, edge deployment) would you prefer each method?

3.5 [Writeup] Possible Reasons Behind Observations (5 points)

1. Why can LoRA with a small rank capture most of the performance of full fine-tuning? What does this imply about the intrinsic dimensionality of the fine-tuning update?
2. Why might BitFit (training only biases) work surprisingly well for NLI?

3. If increasing LoRA rank beyond a certain point shows no improvement, what does that suggest about the task complexity?

4 Cross-Dataset Generalisation

4.1 Setup

In this task you will evaluate how well your SNLI-trained model transfers to different NLI datasets without any additional training. Use your best model from Task 1. Do **not** retrain.

4.2 [Code] Perform Task (5 points)

Load MultiNLI (both the matched and mismatched validation splits) and ANLI (at least Round 1). Verify that label mappings are consistent across datasets (0 = entailment, 1 = neutral, 2 = contradiction). Tokenize each dataset with the same preprocessing as Task 1 and run zero-shot evaluation (inference only, no training).

```
from datasets import load_dataset
from sklearn.metrics import accuracy_score

multinli = load_dataset("multi_nli")
anli = load_dataset("anli")

# Tokenize MultiNLI matched split
multinli_tok = multinli["validation_matched"].map(
    tokenize_fn, batched=True)

# Zero-shot evaluation
predictions = trainer.predict(multinli_tok)
preds = np.argmax(predictions.predictions, axis=-1)
acc = accuracy_score(predictions.label_ids, preds)
print(f"MultiNLI Matched Accuracy: {acc:.4f}")
```

Evaluate on: SNLI test, MultiNLI matched, MultiNLI mismatched, and ANLI Round 1.

4.3 [Code, Writeup] Analysis and Plots (5 points)

Create a bar chart of accuracy across all four evaluation datasets. Generate confusion matrices for the two datasets where accuracy is lowest. Manually inspect 10 misclassified examples from the worst-performing dataset: record the premise, hypothesis, predicted label, true label, and a one-sentence explanation of the likely failure reason.

4.4 [Writeup] Comparative Study (5 points)

Compare accuracy on in-domain (SNLI) vs. out-of-domain datasets. How large is the gap? Compare MultiNLI matched vs. mismatched: which is harder and why? Categorise your 10 error examples into types: *world knowledge required*, *complex syntax*, *lexical overlap trap*, *ambiguous gold label*. Which category dominates?

4.5 [Writeup] Possible Reasons Behind Observations (5 points)

1. Why does accuracy drop sharply on ANLI compared to MultiNLI? Consider how each dataset was constructed.

2. SNLI premises come from image captions. How might this domain bias affect transfer to MultiNLI, whose premises come from varied genres?

3. Propose two concrete strategies to improve cross-dataset robustness. Explain the reasoning behind each.

5 Data Efficiency Analysis

5.1 Setup

In this task you will investigate how much labelled training data is needed for effective fine-tuning. You will train on subsets of SNLI of varying sizes and measure the effect on dev accuracy.

5.2 [Code] Perform Task (5 points)

Sample random subsets of the SNLI training set at sizes 1%, 10%, 25%, 50%, and 100%. For each size, run the experiment. Reload the pre-trained model from scratch for every run.

```
seeds = [42]
fractions = [0.01, 0.10, 0.25, 0.50, 1.0]

for frac in fractions:
    for seed in seeds:
        n = int(len(dataset["train"]) * frac)
        subset = dataset["train"].shuffle(
            seed=seed).select(range(n))
        model = AutoModelForSequenceClassification \
            .from_pretrained(
                "distilbert-base-uncased", num_labels=3)
        # Train with baseline hyperparameters
        # Record dev accuracy
```

This requires 5 training runs. Plan your compute time accordingly.

5.3 [Code, Writeup] Analysis and Plots (5 points)

Compute the mean and standard deviation of accuracy for each fraction. Plot a learning curve with error bars: fraction of data (log scale) on the x-axis, dev accuracy on the y-axis.

5.4 [Writeup] Comparative Study (5 points)

Compare accuracy at 10% data vs. 100% data. What fraction of peak performance is retained? Compare variance (error bar size) at 1% vs. 50%. What does this tell you about reliability at low-data regimes? Identify the "knee" of the curve—the point of diminishing returns.

5.5 [Writeup] Possible Reasons Behind Observations (5 points)

1. Why does the model still achieve reasonable accuracy with only 5–10% of the data? What role does pre-training play?
2. If you were building an NLI system and had to pay $0.10 per labelled example, how much data would you recommend collecting? Justify your answer using the learning curve.

Grading Rubric

Each task is worth 20 points, broken down as follows:

Subtask	**Points**	**Criteria**
Perform Task	5	Correct implementation, valid results
Analysis and Plots	5	Clear tables, plots, and reported metrics
Comparative Study	5	Thoughtful comparison of methods or results
Possible Reasons	5	Insightful explanations of observed patterns

Submission Guidelines

- Submit code as Jupyter notebooks.
- Include all plots, tables, and confusion matrices in separate writeup pdf.
- For each task, write a short analysis (∼half a page) covering the analysis, comparative study, and reasoning subtasks in same writeup pdf.
- Grading emphasis: your reasoning matters more than achieving high accuracy numbers.